Document Management System

Technical Architecture & Demo Guide

Spring Boot · MongoDB GridFS · AWS S3 · OCR · Scheduled Migration

Version	Status	Organization	Module
1.0.0	Demo Ready	Mahadiscom	doc-management

Table of Contents

1. System Overview
2. Technology Stack
3. System Architecture
4. Core Components
5. Document Upload Flow
6. OCR Pipeline
7. Download & Storage Routing
8. Nightly Migration Scheduler
9. REST API Reference
10. Entity & Enum Reference
11. Exception Handling
12. Demo Script & FAQ

1. System Overview

The Document Management System is a Spring Boot microservice that provides secure document upload, OCR-based text extraction, hybrid cloud storage, and automated cold-storage migration. It is designed to handle PDF and image documents at scale, extracting searchable text asynchronously while the API remains responsive.

> Core value proposition: Upload once — validate, deduplicate, store, index (OCR), and auto-migrate to cost-efficient cold storage — all without any manual intervention.

Key Capabilities

File upload & validation	Accepts PDF, PNG, JPEG · max 10 MB · rejects empty files
Duplicate prevention	SHA-256 hash checked per user before storing
GridFS storage	MongoDB GridFS for initial binary storage in chunks
Async OCR	Tesseract (LSTM) + OpenCV preprocessing on a separate thread
Download routing	Transparent: serves from GridFS or S3 based on age
Nightly migration	Docs older than 30 days automatically moved to AWS S3
Structured metadata	All document properties persisted in PostgreSQL via JPA
Error handling	Structured JSON errors for all failure scenarios

2. Technology Stack

Layer	Technology	Purpose
Framework	Spring Boot 3.x	Application server, DI, REST
Persistence	Spring Data JPA / Hibernate	DocumentMetadata ORM
Database	PostgreSQL	Relational metadata storage
File store	MongoDB GridFS	Binary chunk storage (local)
Cloud store	AWS S3 SDK v2	Cold storage after migration
OCR engine	Tesseract4J (LSTM mode)	Text extraction from images
Vision	OpenCV (nu.pattern)	Image preprocessing before OCR
PDF render	Apache PDFBox	Render PDF pages to BufferedImage
Mapping	ModelMapper	Entity ↔ DTO conversion
Async	@Async + ThreadPoolTaskExecutor	Non-blocking OCR
Scheduler	@Scheduled cron	Nightly migration job
Validation	Custom FileValidator	Type, size, empty checks
Dedup	SHA-256 (HashUtil)	Content-based duplicate detection

3. System Architecture

The system follows a layered architecture with four clearly separated tiers. Each layer communicates only with the layer directly below it, ensuring clean separation of concerns and independent testability.

CLIENT LAYER	HTTP consumers — REST client, browser, Postman
API LAYER	DocumentController — validates, routes, responds
SERVICE LAYER	DocumentService · OcrService · GridFsService · MigrationService · S3Service
STORAGE LAYER	PostgreSQL (metadata) · MongoDB GridFS (binary) · AWS S3 (cold)

Cross-Cutting Concerns

FileValidator	Type whitelist (PDF/PNG/JPEG), 10 MB limit, empty-file check
HashUtil	SHA-256 fingerprint for every uploaded file
GlobalExceptionHandler	Unified JSON error responses across all controllers
OpenCVUtil	BufferedImage → OpenCV Mat conversion helper
DocumentMetadataService	Centralized metadata builder and persistence

4. Core Components

DocumentController	Single REST controller exposing all three endpoints. Delegates to DocumentService for all business logic. Returns DocumentMetadataResponse DTOs.
DocumentService	Orchestrates the full upload pipeline: validation → hash → dedup check → GridFS save → metadata save → async OCR trigger. Also handles download routing and document listing.
OcrService	Annotated @Async. Converts the temp file to BufferedImage (using PDFBox for PDFs), preprocesses with OpenCV, runs Tesseract LSTM, cleans the text, saves back to PostgreSQL.
GridFsService	Wraps Spring Data MongoDB GridFsTemplate. Stores files as ObjectId-keyed binary chunks. Retrieves by ObjectId. Deletes after successful S3 migration.
S3Service	Wraps AWS SDK v2 S3Client. Uploads byte arrays with a UUID-prefixed key. Downloads as InputStream for streaming to the client.
MigrationService	Paginates over documents older than 30 days with storageType=LOCAL and migrationStatus=PENDING. Processes in parallel via a 5-thread ExecutorService. On success: sets S3 key, updates storageType and migrationStatus, deletes GridFS copy.
SchedulerService	Wraps MigrationService behind @Scheduled(cron='0 0 2 * * ?'). Fires every night at 02:00. Logs start time and delegates immediately.
DocumentMetadataService	Builds and persists the DocumentMetadata entity. Sets defaults: storageType=LOCAL, migrationStatus=PENDING, ocrStatus=PENDING, documentType=USER_UPLOADED.

5. Document Upload Flow

The upload flow is the most complex path in the system. It combines synchronous validation and storage with asynchronous OCR to keep the API response fast.

1	**POST /user/upload** Client sends multipart file + userId query param
2	**FileValidator.validate()** Rejects empty files, files > 10 MB, non-PDF/PNG/JPEG types
3	**HashUtil.generateHash()** Computes SHA-256 over raw file bytes
4	**Duplicate check** findByFileHashAndUserId() — throws 409 if found
5	**GridFsService.store()** Streams file into MongoDB as binary chunks; returns ObjectId
6	**DocumentMetadataService.save()** Persists metadata row: storageType=LOCAL, ocrStatus=PENDING
7	**documentType == USER_UPLOADED?** Conditional — SYSTEM_GENERATED files skip OCR
8	**File.createTempFile()** Writes MultipartFile to disk — required by Tesseract Java API
9	**@Async OcrService.extractTextAsync()** Runs on a separate thread — API returns immediately
10	**Return 200 + DocumentMetadataResponse** Caller gets metadata; OCR completes in background

Important: The temp file is always deleted in the finally block of extractTextAsync(), ensuring no disk leak even when OCR throws an exception.

6. OCR Pipeline

Text extraction runs asynchronously on a background thread. The pipeline branches on file type: PDFs are rendered page-by-page using PDFBox at 300 DPI before being passed to OpenCV. Plain images are read directly.

1	**Branch on file type** PDF → PDFBox renders each page at 300 DPI to BufferedImage. Image → ImageIO.read() directly.
2	**OpenCVUtil.bufferedImgToMat()** Converts BufferedImage (TYPE_3BYTE_BGR) to OpenCV Mat for processing.
3	**Imgproc.cvtColor()** Converts BGR colour Mat to grayscale — reduces noise and standardises input.
4	**Imgproc.GaussianBlur()** 5x5 Gaussian kernel smooths out scanning artefacts and minor noise.
5	**Imgproc.adaptiveThreshold()** ADAPTIVE_THRESH_GAUSSIAN_C with blockSize=11 — binarises locally to handle uneven lighting.
6	**Imgcodecs.imencode() → ImageIO** Encodes processed Mat back to PNG byte array for Tesseract Java API.
7	**tesseract.doOCR(bufferedImage)** LSTM engine (OEM=1), automatic page segmentation (PSM=1) — runs per page.
8	**cleanOcrText(raw)** Strips non-alphanumeric noise, collapses multiple spaces, removes blank lines.
9	**isOcrFailed(text)** Fails if text is null / under 20 chars / less than 50% alphanumeric content.
10	**Save to PostgreSQL** Sets extractedText, cleanExtractedText, ocrStatus=COMPLETED on the metadata entity.

Tesseract Configuration

datapath	Configured via application property tesseract.datapath
language	Configured via application property tesseract.language (e.g. eng)
PSM (page seg)	Mode 1 — automatic page segmentation with OSD
OEM (engine)	Mode 1 — LSTM neural network engine

7. Download & Storage Routing

The download endpoint is storage-transparent: the caller always uses the same URL regardless of whether the file is in GridFS or S3. Routing logic lives in DocumentService.

1. **GET /user/{documentId}/download**
   userId as query param, documentId in path
2. **findById(documentId)**
   Loads DocumentMetadata from PostgreSQL — throws 404 if not found
3. **Ownership check**
   metadata.getUserId().equals(userId) — throws 401 if mismatch
4. **Branch on storageType == LOCAL**
   Routes to GridFsService.getFile(gridFsId)
5. **Branch on storageType == S3**
   Routes to S3Service.downloadFile(s3Key) → InputStream
6. **Return Resource + headers**
   Content-Disposition: attachment · Content-Type from metadata

Security note: The userId ownership check ensures users cannot download each other's documents by guessing a documentId. The check runs before any storage call.

8. Nightly Migration Scheduler

To reduce MongoDB storage costs, documents older than 30 days are automatically migrated to AWS S3 cold storage. The migration is fault-tolerant: any failure marks the document as FAILED without touching the GridFS copy, so it will be retried the next night.

1	**Cron trigger 0 0 2 * * ?** Fires every night at 02:00 — low-traffic window
2	**Page query (batch = 10)** Finds LOCAL + PENDING docs with uploadedAt < now - 30 days
3	**ExecutorService.submit() x5** 5-thread pool processes documents in parallel per batch
4	**Set status = IN_PROGRESS** Prevents double-processing if scheduler fires twice
5	**GridFS → byte[]** getContentAsByteArray() reads full binary from MongoDB
6	**S3Service.uploadFile()** UUID-prefixed key; PutObjectRequest → S3 bucket
7	**Update metadata** Sets s3Key, storageType=S3, migrationStatus=COMPLETED
8	**GridFsService.deleteFile()** Removes GridFS entry only after confirmed S3 upload
9	**On exception: FAILED** migrationStatus=FAILED; GridFS copy untouched for next retry
10	**Next page** Continues paginating until no more qualifying records

9. REST API Reference

Method	Endpoint	Params	Response
POST	`/user/upload`	file (multipart), userId (query)	200 DocumentMetadataResponse
GET	`/user/{userId}/documents`	userId (path)	200 List<DocumentMetadataResponse>
GET	`/user/{documentId}/download`	userId (query), documentId (path)	200 File stream (Content-Disposition attachment)

Error Response Format

All error responses follow a consistent JSON structure:

Field	Type	Example
status	int	400 / 401 / 404 / 500
error	String	Validation Error
message	String	File size exceeds 10MB limit
timestamp	LocalDateTime	2024-01-15T02:30:00

10. Entity & Enum Reference

DocumentMetadata Entity

Field	Type	Description
documentId	int (PK, SEQ)	Auto-generated primary key via sequence generator
userId	String	Identifies the document owner
fileName	String	Original filename from MultipartFile
fileType	String	MIME type (application/pdf, image/png, image/jpeg)
fileSize	long	File size in bytes
gridFsId	String	MongoDB ObjectId of the stored binary
s3Key	String	UUID-prefixed S3 object key (null until migrated)
uploadedAt	LocalDateTime	Timestamp used for the 30-day migration cutoff
extractedText	TEXT	Raw OCR output (may contain noise characters)
cleanExtractedText	TEXT	Cleaned OCR output after cleanOcrText()
documentType	Enum	USER_UPLOADED or SYSTEM_GENERATED
OcrStatus	Enum	PENDING / COMPLETED / FAILED / SKIP
storageType	Enum	LOCAL (GridFS) or S3
migrationStatus	Enum	PENDING / IN_PROGRESS / COMPLETED / FAILED
fileHash	String	SHA-256 hex string for deduplication

Enums

StorageType	LOCAL — stored in MongoDB GridFS S3 — migrated to AWS S3
MigrationStatus	PENDING → IN_PROGRESS → COMPLETED FAILED — retried next nightly run
OcrStatus	PENDING → COMPLETED FAILED — OCR error SKIP — not applicable
DocumentType	USER_UPLOADED — triggers OCR SYSTEM_GENERATED — skips OCR

11. Exception Handling

All exceptions are caught by GlobalExceptionHandler (@RestControllerAdvice) and converted to structured ErrorResponse JSON. This ensures no stack traces reach the client and all errors are consistently shaped.

Exception	HTTP Status	Trigger
FileValidationException	400 Bad Request	Empty file, wrong type, size > 10 MB
DocumentProcessingException	500 Internal Server Error	Duplicate file detected, processing failure
DocumentNotFoundException	404 Not Found	documentId not in PostgreSQL
UnauthorizedException	401 Unauthorized	userId does not own the document
Exception (catch-all)	500 Internal Server Error	Any unhandled exception

12. Demo Script & Frequently Asked Questions

5-Point Demo Walkthrough

1. Upload a PDF

Call POST /user/upload with a PDF file and userId. Show: (a) a file over 10 MB is rejected with 400, (b) uploading the same file twice returns 409 with 'Duplicate not allowed', (c) a valid upload returns 200 with metadata including ocrStatus=PENDING.

2. OCR in background

After upload, poll or inspect the database to show extractedText being populated asynchronously. Demonstrate that the upload API returned immediately while OCR was still running.

3. List documents

Call GET /user/{userId}/documents to show the list of all documents for a user, including their storageType, ocrStatus, and whether extractedText is populated.

4. Download routing

Download a LOCAL document (storageType=LOCAL) — it streams from GridFS. Then download a document that has been migrated (storageType=S3) — it streams from S3. Same API endpoint, transparent routing.

5. Migration

Trigger processOldDocuments() manually (or show a DB record older than 30 days). Step through: IN_PROGRESS → S3 upload → COMPLETED → GridFS deleted. Show FAILED rollback for a simulated S3 error.

Frequently Asked Questions

Why MongoDB GridFS for file storage?

GridFS splits files into 255 KB chunks, bypassing MongoDB's 16 MB document limit. It also allows streaming large files without loading them entirely into memory.

Why not store files in PostgreSQL?

PostgreSQL BYTEA columns are not optimised for large binary streaming. GridFS (and S3 for cold storage) provide better throughput and storage efficiency.

Why SHA-256 and not filename for dedup?

The same filename can contain different content. SHA-256 detects true content-level duplicates regardless of what the file is named.

Why run OCR asynchronously?

Tesseract on a multi-page PDF can take several seconds per page. Running it synchronously would cause the upload API to time out on large documents.

What if S3 upload fails during migration?

The migrationStatus is set to FAILED and the GridFS copy is NOT deleted. The document will be retried automatically the next nightly run.

Can SYSTEM_GENERATED documents be searched?

Not via OCR — the pipeline is skipped. They can still be listed and downloaded, but will not have extractedText unless populated by other means.

How is the 30-day cutoff calculated?

LocalDateTime.now().minusDays(30) is computed at the start of each scheduler run. Documents with uploadedAt before that timestamp qualify for migration.

What happens to the temp file on OCR failure?

The finally block in extractTextAsync() always calls tempFile.delete(), so no disk accumulation occurs even if Tesseract or OpenCV throws an exception.